MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name	ASLAM KHAN		
Subject Name with Code	5CS5-13/5AM5-13/AD5-13 Distributed System		
Semester	5th	**Session**	2025-26 (Odd)
Branch	Computer Science and Engineering	**Batch**	CS/AIDS/AIML
Unit No.	3	**Submission Date**	18-Sep-25

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	Processes & threads; multithreading in DS.	1-9	9
2	Client/Server model, RPC, RMI (basics)	10-14	5
3	Message-oriented communication, multicast basics, time services.	15-18	4
4	Language mechanisms for synchronization (monitors, semaphores in DS context)	19-22	4
5	Distributed process scheduling – model & metrics.	23-27	25
6	Distributed File Systems (DFS): transparency goals, naming, key characteristics.	28-33	6
7	DFS design & implementation overview (e.g., NFS, AFS)	34-47	14

Faculty

HOD

Dean Academics 18/9/25

References/Text Books	
1	Distributed Systems: Principles and Paradigms By Andrew S. Tanenbaum & Maarten Van Steen
2	Distributed Systems: Concepts and Design By George Coulouris, Jean Dollimore, Tim Kindberg, Gordon Blair
3	Designing Data-Intensive Applications By Martin Kleppmann

UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

CHALLENGE OF GLOBAL TIME

LECTURE NO: -1

Introduction: Concurrency in Distributed Systems

In distributed systems, multiple processes can execute concurrently across different nodes. This concurrency introduces both challenges and opportunities. This chapter will delve into the fundamental concepts of concurrent processes, the models used to represent them, and the mechanisms employed to ensure their correct and synchronized operation.

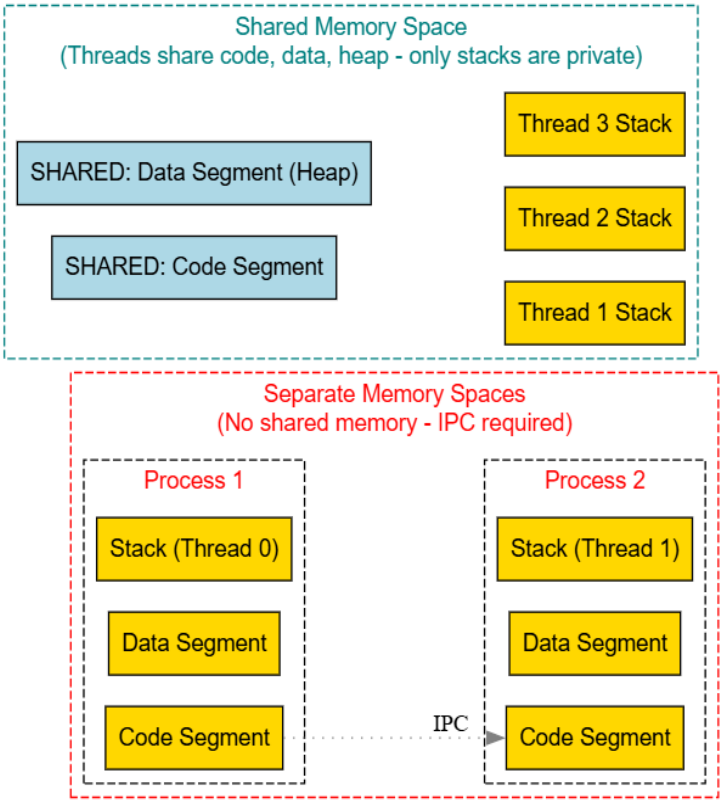
Processes and Threads

Processes

A process is an instance of a computer program that is being executed. It is an independent unit with its own virtual address space, program counter, stack, and data. Processes are isolated from one another, and communication between them requires specific mechanisms like inter-process communication (IPC). A process can be further broken down into several parts:

- **Text Section:** The executable code.
- **Data Section:** Global variables.
- **Heap:** Dynamically allocated memory.
- **Stack:** Temporary data (function parameters, return addresses, local variables).
- **Process Control Block (PCB):** Contains process state (running, waiting, etc.), program counter, CPU registers, memory management information, and other scheduling data.

Process Memory Layout A simple block diagram showing the different parts of a process's memory space. It would be a vertical stack of blocks, with the "Text Section" at the bottom, followed by the "Data Section", "Heap" (growing upwards), and "Stack" (growing downwards), with the "Command Line Arguments" and "Environment Variables" at the very top. This visually represents the logical layout of a process in memory.



Threads

A thread, often called a lightweight process, is a single sequence of instructions within a process. Multiple threads can exist within the same process and share its resources, including memory space and open files. This shared-memory model makes communication and data exchange between threads much more efficient than between processes.

Key Distinction:

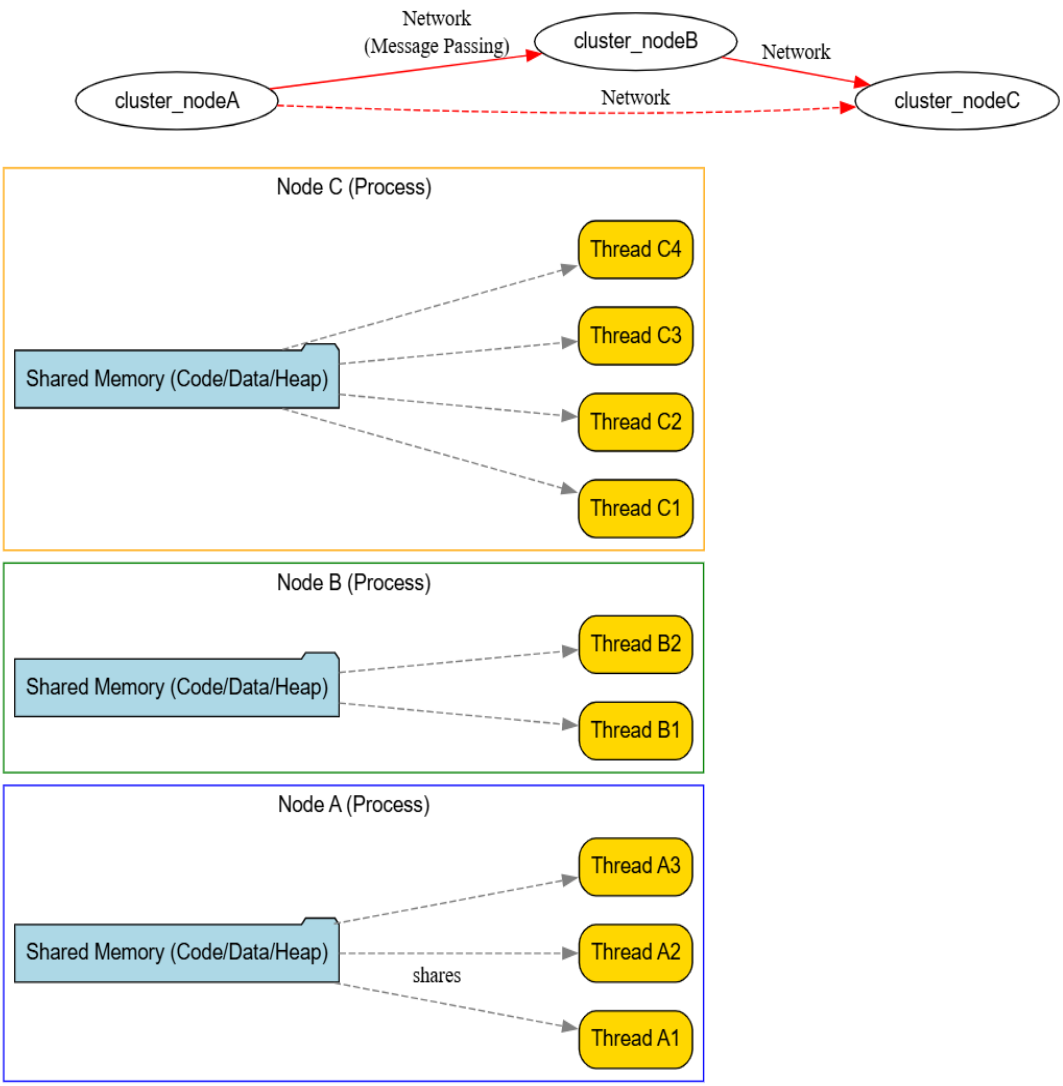
Feature	**Processes**	**Threads**
Independence	Independent units	Sub-units of a process
Address Space	Separate address spaces	Shared address space
Overhead	High creation/context-switching overhead	Low creation/context-switching overhead
Communication	IPC mechanisms (e.g., pipes, message queues)	Shared memory

Processes vs. Threads This diagram would show two separate, distinct rectangles representing two processes. Each rectangle would contain its own text, data, heap, and stack sections, visually emphasizing their isolation. Below this, a single large rectangle would represent one process, containing a shared text, data, and heap section. Within this larger rectangle, two or more smaller, distinct stack sections would represent individual threads, clearly showing that they share the process's resources but have their own execution contexts.

Multithreading in Distributed Systems:

In the realm of modern computing, distributed systems (DS) have become the backbone of scalable and efficient applications, ranging from cloud computing to global communication networks.

A distributed system comprises multiple independent computers (nodes) that work together to achieve a common goal, communicating over a network. One of the key techniques enhancing the performance and responsiveness of these systems is **multithreading**.



Multithreading involves executing multiple threads—lightweight subprocesses within a single process—concurrently to handle various tasks. This essay explores the role of multithreading in distributed systems, its benefits, challenges, implementation strategies, and real-world applications, highlighting its significance in today's interconnected world.

Understanding Multithreading and Distributed Systems

A **thread** is the smallest unit of processing that can be scheduled by an operating system. Unlike processes, which have separate memory spaces, threads within the same process share memory, enabling faster communication and resource utilization. Multithreading extends this concept by allowing multiple threads to run simultaneously within a process, leveraging the capabilities of multi-core processors.

A **distributed system**, on the other hand, is a collection of autonomous nodes—computers or servers—located at different geographical or logical locations. These nodes collaborate to perform tasks, relying on message passing or remote procedure calls (RPC) for communication. The integration of multithreading in such systems enables each node to handle multiple operations concurrently, improving throughput and responsiveness. For instance, while one thread processes a client request, another might fetch data from a remote node, optimizing resource use.

Benefits of Multithreading in Distributed Systems

Multithreading offers several advantages that make it indispensable in distributed environments:

1. **Parallel Execution**: Multithreading allows tasks to be divided among multiple threads running on different CPU cores. In a distributed system, this parallelism ensures that nodes can process multiple requests or computations simultaneously, significantly boosting performance. For example, a web server with multithreading can handle hundreds of client connections concurrently.
2. **Improved Responsiveness**: By assigning different threads to handle input/output (I/O) operations, computation, and network communication, multithreading ensures that the system remains responsive. If one thread is blocked waiting for a network response, others can continue executing, preventing delays.
3. **Resource Sharing**: Threads within a process share the same memory space, reducing the overhead of inter-process communication (IPC). In a distributed system, this shared memory facilitates efficient data exchange between threads on the same node, while message passing handles inter-node communication.
4. **Scalability**: As the workload increases, additional threads can be spawned to distribute tasks across nodes. This scalability is crucial for distributed systems like cloud platforms, where demand fluctuates dynamically.
5. **Cost Efficiency**: Multithreading maximizes the utilization of existing hardware by leveraging multi-core processors, reducing the need for additional nodes and lowering operational costs.

Challenges of Multithreading in Distributed Systems

Despite its benefits, multithreading in distributed systems poses several challenges that must be addressed:

1. **Synchronization Issues**: Since threads share memory, concurrent access to shared resources can lead to **race conditions**, where the outcome depends on the unpredictable order of thread execution. Synchronization mechanisms like locks,

semaphores, and monitors are required, but improper use can cause **deadlocks**—a state where threads wait indefinitely for each other.

2. **Complexity**: Managing multiple threads across distributed nodes increases system complexity. Developers must ensure thread safety, handle thread lifecycle (creation, termination), and coordinate between nodes, which can introduce bugs if not carefully designed.
3. **Network Delays**: In a distributed system, threads rely on network communication, which introduces latency. A thread waiting for a remote response may hinder performance, necessitating techniques like asynchronous I/O or thread pooling.
4. **Resource Contention**: Multiple threads competing for CPU, memory, or network resources can lead to contention, reducing overall efficiency. This is particularly challenging in distributed environments where resources are distributed across nodes.
5. **Fault Tolerance**: A thread failure on one node can affect the entire system if not isolated. Distributed systems require robust error handling to ensure reliability, adding another layer of complexity.

Implementation Strategies

Implementing multithreading in distributed systems requires careful design and the use of appropriate tools and techniques:

1. **Thread Creation and Management**: Programming languages like Java, C++, and Python provide libraries (e.g., java.lang.Thread, pthread, threading) to create and manage threads. In a distributed context, thread pools—pre-allocated sets of threads—can be used to limit the number of active threads and improve resource management.
2. **Synchronization Mechanisms**:
   - **Locks**: Ensure mutual exclusion by allowing only one thread to access a resource at a time (e.g., pthread_mutex_lock in C).
   - **Semaphores**: Control access to a finite number of resources.
   - **Monitors**: Encapsulate synchronization logic, as seen in Java's synchronized blocks, providing a higher-level abstraction. These mechanisms prevent data corruption but must be used judiciously to avoid deadlocks.
3. **Message Passing**: Threads on different nodes communicate using protocols like TCP/IP or RPC. Asynchronous messaging (e.g., using callbacks or futures) allows threads to continue other tasks while waiting for responses, enhancing efficiency.
4. **Load Balancing**: Distributing threads across nodes based on workload ensures optimal resource utilization. Load balancers or distributed task queues (e.g., RabbitMQ) can assign tasks to the least busy threads.

5. **Error Handling**: Implementing try-catch blocks or timeout mechanisms ensures that thread failures do not cascade across the system. Distributed systems often use replication or failover strategies to maintain availability.

Real-World Applications

Multithreading in distributed systems is widely applied in various domains:

1. **Web Servers**: Apache and Nginx use multithreading to handle multiple client requests concurrently, distributing them across threads on different server nodes.
2. **Cloud Computing**: Platforms like Amazon Web Services (AWS) and Google Cloud use multithreading to manage virtual machines and storage services, with threads handling user requests and data synchronization across data centers.
3. **Distributed Databases**: Systems like Cassandra or MongoDB employ multithreading to process queries and replicate data across nodes, ensuring high availability and performance.
4. **Online Gaming**: Multiplayer games use multithreading to manage player interactions, physics calculations, and network updates across distributed servers.
5. **Big Data Processing**: Frameworks like Apache Hadoop and Spark use multithreading within nodes to parallelize data processing tasks across a distributed cluster.

Case Study: A Distributed File System

Consider a distributed file system like Google File System (GFS). Each chunk server runs multiple threads: one thread handles client read requests, another writes data to disk, and a third communicates with the master node for metadata updates. Multithreading ensures that I/O operations and network communication occur concurrently. Synchronization is achieved using locks to prevent concurrent writes to the same chunk, while condition variables allow threads to wait for data availability. This design exemplifies how multithreading enhances performance and reliability in a distributed environment.

Future Trends

The future of multithreading in distributed systems lies in leveraging advanced hardware and software innovations. The rise of multi-core and many-core processors will further emphasize parallel execution. Technologies like **actor models** (e.g., Akka) and **asynchronous programming** (e.g., async/await in C#) are evolving to simplify thread management. Additionally, the integration of artificial intelligence for dynamic thread scheduling and fault prediction will enhance the robustness of distributed systems.

Concurrency vs. Parallelism

- **Concurrency:** The composition of independently executing processes. It is a way of structuring a system to handle multiple tasks at once. In a single-core system, this is achieved by interleaving the execution of tasks.
- **Parallelism:** The simultaneous execution of multiple tasks. It requires hardware with multiple processing cores and is used to solve a problem faster by breaking it into smaller parts that can be executed concurrently.

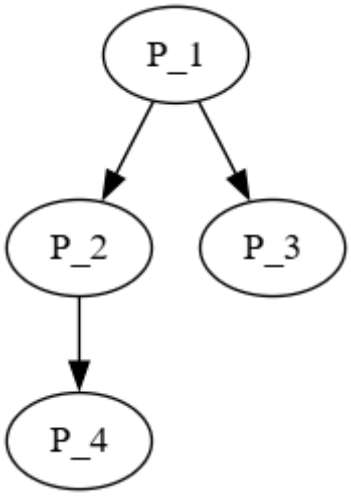
Graph Models for Process Representation

Graph models are used to formally represent the dependencies and interactions between concurrent processes.

Process-Interaction Graphs

A process-interaction graph is a directed graph where nodes represent processes and edges represent the flow of information or synchronization dependencies between them. A directed edge from process P_i to P_j signifies that P_i must send a message or signal to P_j. These graphs are useful for visualizing the overall structure of a distributed application.

Process-Interaction Graph A diagram with nodes labeled P_1,P_2,P_3,P_4. Directed edges could be drawn from P_1→P_2, P_1→P_3, and P_2→P_4. This would visually represent that P_1 sends messages to both P_2 and P_3, and P_2 sends a message to P_4.



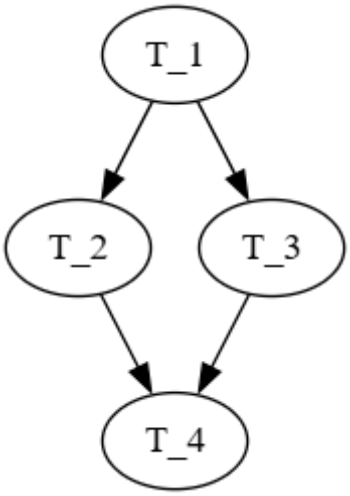
Concurrency Graphs

Concurrency graphs are used to analyze the potential for concurrent execution. A node in the graph represents an operation, and a directed edge indicates a dependency. If there is no path between two nodes, their corresponding operations can be executed concurrently. For example, consider a task to be broken down into four sub-tasks:

- T_1: Read data from file.
- T_2: Process data A.
- T_3: Process data B.
- T_4: Write results to a file.

If T_2 and T_3 both depend on T_1, and T_4 depends on both T_2 and T_3, the concurrency graph would have edges T_1→T_2, T_1→T_3, T_2→T_4, and T_3→T_4. Since there is no path between T_2 and T_3, they can be executed concurrently.

Concurrency Graph Example A diagram with four nodes representing tasks T_1,T_2,T_3,T_4. A directed edge would go from T_1 to both T_2 and T_3, showing they depend on T_1. Two directed edges would then go from T_2→T_4 and T_3→T_4, showing that T_4 depends on both. The parallel paths from T_1 to T_4 through T_2 and T_3 would visually highlight the concurrency of tasks T_2 and T_3.



UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

CLIENT/SERVER MODEL, RPC, RMI

LECTURE NO: -2

Client/Server Model

The **Client/Server Model** is a fundamental architecture in distributed systems where tasks are divided between **clients** and **servers**. A client is an entity that requests services or resources, while a server is an entity that provides those services, such as data storage or computation. Communication occurs over a network using protocols like TCP/IP or HTTP, making it a cornerstone of concurrent systems where multiple clients can interact with a server simultaneously.

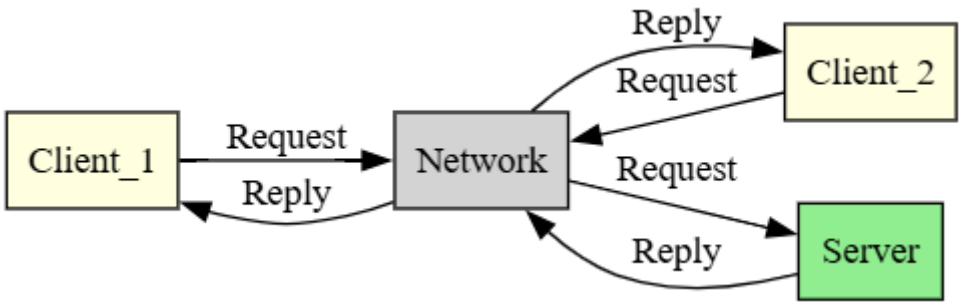
- **Concurrency in Client/Server**: The model supports concurrency by allowing multiple client processes to send requests concurrently. The server, often multi-threaded or multi-processed, handles these requests in parallel, leveraging operating system support for concurrent execution. For instance, a web server might use a thread pool to manage hundreds of client connections.
- **Key Features**:
  - **Asymmetry**: Clients initiate requests, and servers respond.
  - **Centralized Control**: The server manages resources, simplifying data consistency.
  - **Scalability**: Additional servers can be added to balance loads.
- **Advantages**: Efficient resource use, centralized management, and ease of scaling.
- **Limitations**: A single point of failure (server crash) and potential bottlenecks with high client loads.
- **Application in Concurrent Programming**: In a concurrent setup, the server implements thread-based or process-based models to handle multiple client requests without blocking, ensuring responsiveness. For example, an email server processes incoming messages from various clients concurrently.

Stateless vs. Stateful Servers:

- **Stateless Server:** Does not maintain any information about the client's past requests. Each request is handled independently. This is scalable but requires clients to send all necessary information with every request.

- **Stateful Server:** Maintains information about the client's state across multiple requests. This can be more efficient for some applications but is more complex to implement and manage, especially in the event of a server crash.

Client/Server Interaction: A diagram showing one or more client machines on the left. A network cloud or line in the middle. A server machine on the right. Arrows would go from the clients, through the network, to the server, labeled "Request." Corresponding arrows would go from the server, through the network, back to the clients, labeled "Reply." This would clearly illustrate the fundamental request-reply pattern.



Inter-Process Communication (IPC)

Processes communicate with each other to exchange data and coordinate their activities. In a distributed system, this often happens across the network.

- **Message Passing:** Processes communicate by explicitly sending and receiving messages. This is the most common form of IPC in distributed systems.
- **Remote Procedure Call (RPC):** Allows a program to cause a procedure (subroutine) to execute in another address space (commonly on another computer) without the programmer explicitly coding the remote interaction. It simplifies distributed programming by making remote calls appear like local ones.
- **Remote Method Invocation (RMI):** A Java-specific protocol that allows an object running in one Java Virtual Machine (JVM) to invoke a method on an object running in another JVM.

Remote Procedure Call (RPC)

Remote Procedure Call (RPC) is a communication mechanism that allows a client to execute a procedure or function on a remote server as if it were a local call, abstracting the network complexity. It is a critical tool in concurrent programming for enabling distributed processes to interact seamlessly.

- **Concurrency in RPC**: RPC supports concurrency by allowing multiple client threads to invoke remote procedures simultaneously. The server uses a thread or process per request to handle these calls concurrently, ensuring efficient resource utilization. This aligns with the principles of Unit 3, where multiple processes or threads execute in parallel.
- **How It Works**:
    1. **Client Stub**: Marshals (serializes) the procedure call and parameters into a message.
    2. **Network Transmission**: The message is sent over the network.
    3. **Server Stub**: Unmarshals the message and executes the procedure.
    4. **Result Return**: The result is marshaled back to the client.
- **Key Features**:
    - **Transparency**: Hides network details, making remote calls appear local.
    - **Synchronous**: The client waits for the server's response, which can be a concurrency bottleneck if not managed.
- **Advantages**: Simplifies distributed programming and reuses local call semantics.
- **Limitations**: Network latency and potential failures (e.g., server unavailability) can disrupt concurrent operations.
- **Application in Concurrent Programming**: RPC is used in distributed file systems where a client thread requests file data, and a server thread processes it concurrently. Synchronization mechanisms like locks ensure thread safety during remote calls.

Remote Method Invocation (RMI)

Remote Method Invocation (RMI) is an object-oriented extension of RPC, designed for languages like Java, allowing a client to invoke methods on a remote object as if it were local. It is particularly relevant to concurrent programming in distributed systems.

- **Concurrency in RMI**: RMI supports concurrency by enabling multiple client threads to invoke methods on remote objects simultaneously. The server uses a thread pool to handle these requests, aligning with Unit 3's focus on managing concurrent processes. This object-based approach enhances flexibility in distributed applications.
- **How It Works**:
    1. **Remote Interface**: Defines methods available for remote invocation.
    2. **Stub and Skeleton**: The client uses a stub, and the server uses a skeleton to manage communication.
    3. **Object Serialization**: Parameters and return values are serialized for network transmission.
    4. **Registry**: The RMI Registry helps the client locate the remote object.
- **Key Features**:

  - **Object-Oriented**: Supports complex Java objects.
  - **Distributed Garbage Collection**: Manages remote object references automatically.
  - **Asynchronous Option**: Allows non-blocking calls via callbacks.
- **Advantages**: Seamless integration with Java's object model and robust handling of distributed objects.
- **Limitations**: Limited to Java environments and introduces overhead due to serialization.
- **Application in Concurrent Programming**: RMI is used in distributed enterprise applications where a client thread invokes a method on a remote business object, with the server handling concurrent requests using threads.

Comparative Analysis

The Client/Server Model, RPC, and RMI serve distinct yet interrelated roles in concurrent programming within distributed systems:

- **Client/Server Model**: Provides the architectural framework, enabling concurrency through multiple client-server interactions. It is the broadest concept, setting the stage for communication.
- **RPC**: Builds on this by offering a procedural interface for remote execution, supporting concurrency through multi-threaded servers. It is less object-oriented than RMI.
- **RMI**: Extends RPC with object-oriented capabilities, enhancing concurrency by allowing method calls on remote objects, ideal for Java-based concurrent systems.
- **Commonality**: All three rely on network communication and benefit from concurrent execution to improve performance.
- **Differences**: The model is structural, RPC is procedural, and RMI is object-based, with varying levels of abstraction and language support.

Challenges and Synchronization

- **Race Conditions**: Multiple threads accessing shared resources (e.g., in RPC or RMI) can lead to inconsistent data. Synchronization tools like locks or monitors are essential.
- **Deadlocks**: Improper lock management in concurrent RPC or RMI calls can cause threads to wait indefinitely.
- **Network Latency**: Delays in Client/Server communication or RPC/RMI calls require asynchronous techniques to maintain concurrency.
- **Solution**: Use thread pools, semaphores, or condition variables to manage concurrent access and ensure thread safety.

Practical Applications

- **Client/Server Model**: Web servers (e.g., Apache) handle concurrent HTTP requests from multiple clients using threaded or forked processes.
- **RPC**: Distributed file systems (e.g., NFS) use RPC to allow concurrent file access across nodes.
- **RMI**: Java-based distributed applications (e.g., enterprise resource planning systems) leverage RMI for concurrent object interactions across servers.

UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

MESSAGE-ORIENTED COMMUNICATION, MULTICAST AND TIME SERVICES

LECTURE NO: -4

Message-Oriented Communication

Message-Oriented Communication is a model where processes in a distributed system exchange data in the form of **messages** rather than through direct procedure calls. This approach decouples the sender and receiver, allowing asynchronous interaction, which is vital for concurrent systems.

- **Fundamentals**:
  - Messages are sent and stored by an intermediary, such as a message queue or broker, until the receiver is ready to process them.
  - It supports two main paradigms: **Message Queuing** (e.g., RabbitMQ, ActiveMQ), where messages are placed in a queue, and **Publish/Subscribe** (e.g., MQTT, Kafka), where publishers send messages to topics, and subscribers receive based on interest.
- **Concurrency in Role**: In concurrent programming, message-oriented communication allows producer and consumer threads to operate independently. A producer thread can send messages to a queue while a consumer thread processes them asynchronously, preventing blocking and enhancing responsiveness.
- **Advantages**:
  - **Decoupling**: Sender and receiver are independent in time and space.
  - **Reliability**: Persistent queues ensure message delivery even if the receiver is offline.
  - **Scalability**: Handles high message volumes efficiently.
- **Limitations**:
  - Overhead from message brokers.
  - Risk of message loss if persistence is not configured.
- **Application**: In e-commerce, order processing uses message queues where a thread places an order, and another thread fulfills it asynchronously, supporting concurrent user requests.

Multicast Basics

Multicast Basics refer to a communication technique where a single sender transmits data to multiple receivers simultaneously, forming a **multicast group**. This method is optimized for one-to-many distribution in distributed systems.

- **Fundamentals**:
  - The sender transmits to a multicast address (e.g., 224.0.0.0 to 239.255.255.255 in IPv4), and network routers replicate the data to all group members.
  - Receivers join or leave the group dynamically using protocols like IGMP (Internet Group Management Protocol).
  - It typically uses UDP for real-time delivery, though reliability requires additional protocols.
- **Concurrency in Role**: Multicast supports concurrency by allowing multiple nodes to receive and process the same data simultaneously. Threads on different nodes can listen to multicast messages, enabling parallel execution without individual unicast sends.
- **Advantages**:
  - **Bandwidth Efficiency**: Reduces network load compared to sending separate messages.
  - **Real-Time**: Ideal for applications requiring simultaneous updates.
- **Limitations**:
  - Limited network support (not all routers handle multicast).
  - No inherent delivery guarantee, posing reliability challenges.
- **Application**: In online gaming, a server uses multicast to send game state updates to all players concurrently, ensuring synchronized experiences across distributed nodes.

Time Services

In a distributed system, without a single, global clock, establishing a consistent notion of time is a significant challenge.

Logical Clocks

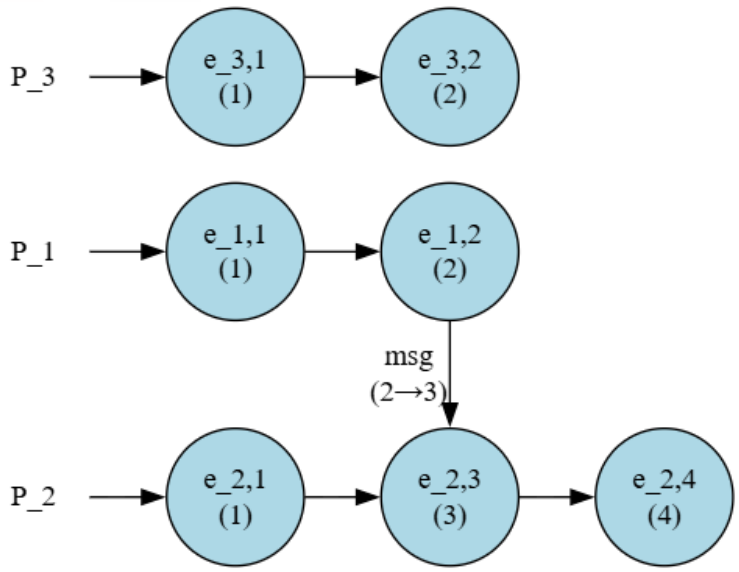
Logical clocks provide a way to order events without relying on synchronized physical time. They capture the causal relationship between events.

- **Lamport's Logical Clocks:** This algorithm defines a "happened-before" relation, denoted as $a \rightarrow b$. If $a \rightarrow b$, then the event a happened before the event b. It assigns a timestamp to each event.
  - If a and b are events in the same process, and a occurs before b, then $C(a) < C(b)$.

- If event a is the sending of a message and b is the receipt of that message, then C(a)<C(b).
- All other events are concurrent.

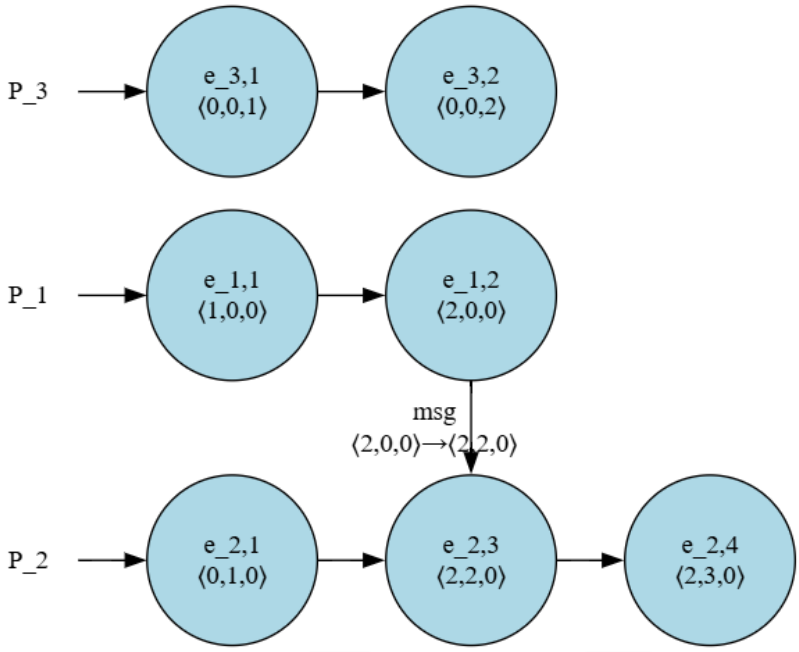
Lamport's Logical Clock Example (Space-Time Diagram) A space-time diagram with three vertical lines representing processes P_1,P_2,P_3. Events would be marked with dots on these lines, and messages would be represented by diagonal arrows between the processes. Each event would be labeled with its Lamport timestamp, showing how the timestamps increase locally and are updated across processes upon message exchange. For example, a message from P_1 (event e_1,2) to P_2 (event e_2,3) would show C(e_1,2)<C(e_2,3), illustrating the happened-before relationship.

- **Vector Clocks:** A vector clock is an array of integers, where each element corresponds to a process in the system. When a process receives a message, it updates its vector clock based on its own timestamp and the timestamp of the received message. Vector clocks can determine if two events are causally related or concurrent. A vector clock VC_i for process P_i is an array of size N (the total number of processes).
  - When an event occurs at P_i, VC_i[i] is incremented.
  - When P_i sends a message, it includes its current VC_i.
  - When P_j receives a message with vector clock VC_msg, it updates its own VC_j by taking the maximum of each element: VC_j[k]=max(VC_j[k],VC_msg[k]) for all k, and then increments its own element: VC_j[j].



Vector Clock Example (Space-Time Diagram) Similar to the Lamport's diagram, this space-time diagram would show processes and messages. However, each event would be labeled with a vector timestamp, e.g., ⟨1,0,0⟩. The diagram would illustrate how these vectors are

updated according to the rules, showing how they can track causal dependencies and identify concurrent events.



Physical Clocks and Clock Synchronization

Physical clocks are the actual clocks on each machine. Clock synchronization protocols are used to keep the physical clocks across a distributed system in sync with a reference time source.

- **Cristian's Algorithm:** A time server sends its time to a client. The client records the round-trip time (RTT) and calculates the server's time. The client then sets its own time to the server's time plus half of the RTT to account for network delay. This algorithm is simple but less precise.
- **Berkeley Algorithm:** A time daemon polls clients to get their clock times. It then calculates an average of all the times and tells each client to adjust their clock (either forward or backward) to this new average. This is more robust as it doesn't rely on a single, authoritative time server.

UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

LANGUAGE MECHANISMS FOR SYNCHRONIZATION

LECTURE NO: -5

Language Mechanisms for Synchronization

When processes or threads share resources, synchronization is crucial to prevent race conditions and ensure data consistency.

Semaphores

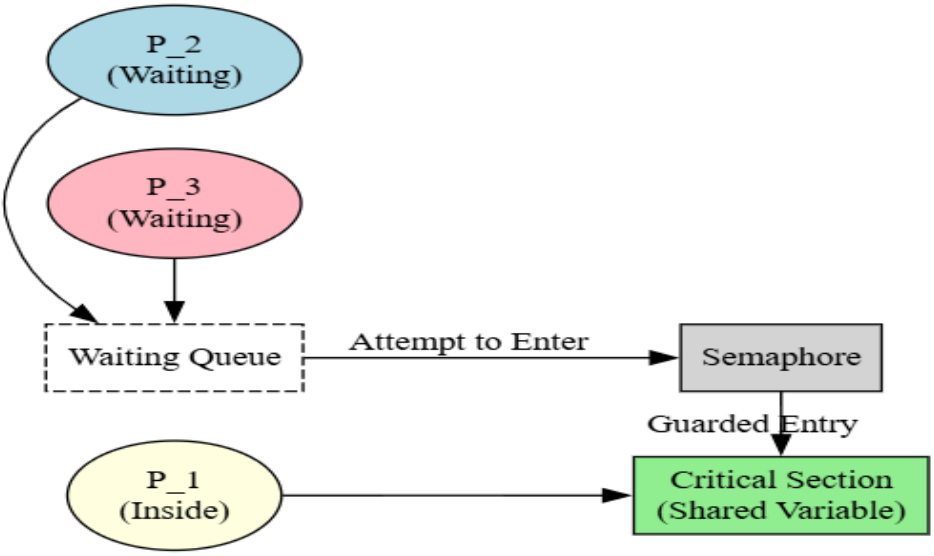
A semaphore is a special integer variable used for signaling among processes. It is a tool for controlling access to a common resource by multiple processes in a concurrent system.

- **P (wait) operation:** Decrements the semaphore value. If the value becomes negative, the process is blocked and added to a queue.
- **V (signal) operation:** Increments the semaphore value. If the value is not positive, it wakes up a waiting process from the queue.

A simple pseudocode example for a binary semaphore controlling access to a critical section:

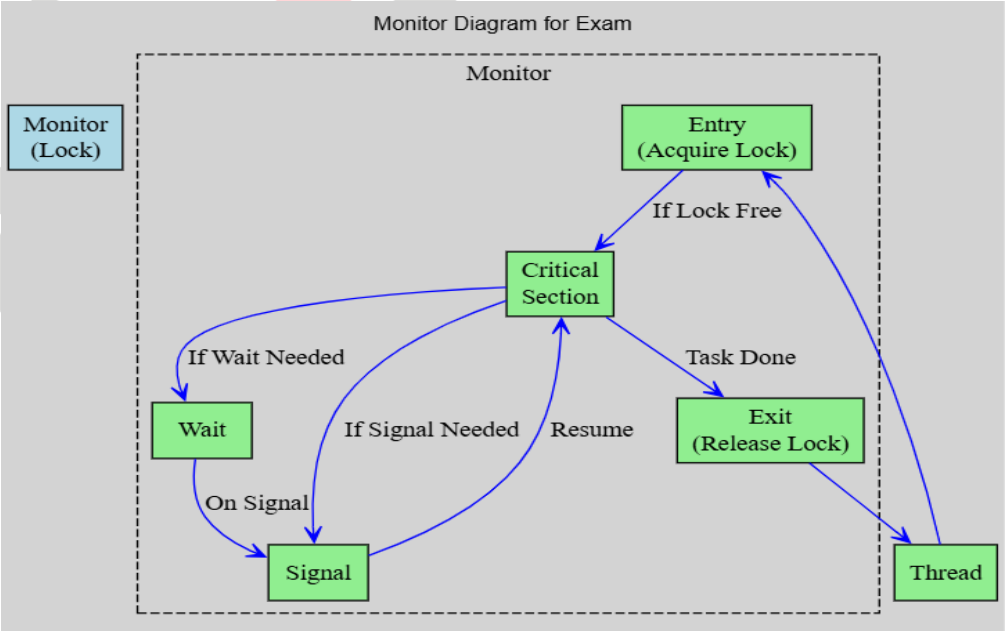
```
semaphore mutex = 1;

process A {
   ...
   P(mutex); // Enter critical section
   // access shared resource
   V(mutex); // Exit critical section
   ...
}
```

Semaphore Operation: A diagram showing a critical section (e.g., a shared variable). An entrance to this section is guarded by a semaphore. Multiple processes would be shown attempting to enter. One process is inside, while others are in a waiting queue, illustrating the mutual exclusion property of a binary semaphore.



Monitors

A monitor is a high-level synchronization construct that encapsulates data and the procedures that operate on that data. It ensures that only one process can be active inside the monitor at any given time, providing mutual exclusion. Monitors often use condition variables to manage waiting and signaling, which allow a process to temporarily release the monitor lock while it waits for a certain condition to be met.



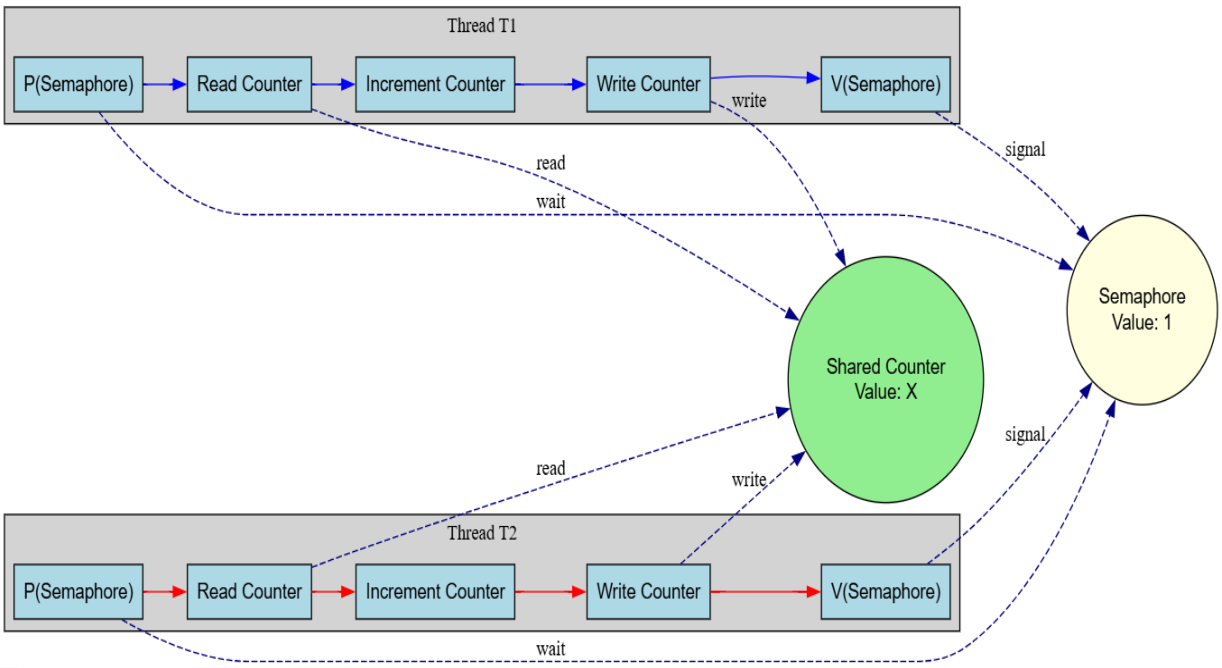
Message Passing

Message passing is a synchronization mechanism where processes communicate by sending and receiving messages.

- **Blocking Send/Receive:** The sender waits until the message is received, and the receiver waits until a message is available. This guarantees a rendezvous.
- **Non-Blocking Send/Receive:** The sender sends the message and continues without waiting. The receiver can check for a message periodically. This approach offers more concurrency but requires careful management to avoid race conditions.

Thread Synchronization with Semaphore

Two threads (T_1, T_2) accessing a shared counter via P/V operations.



Concurrency Control in Distributed Systems

Concurrency control is essential to ensure that concurrent operations on shared data in a distributed database or system produce the same results as if they were executed sequentially.

Distributed Process Scheduling: Models and Metrics

Distributed process scheduling is a critical component of distributed systems, where multiple interconnected computers or nodes collaborate to perform computations. Unlike traditional single-processor scheduling, distributed scheduling involves allocating processes or tasks across geographically dispersed or networked nodes to optimize performance, resource

utilization, and reliability. The goal is to balance workload, minimize delays, and ensure efficient execution in environments like cloud computing, grid systems, and cluster-based architectures.

In distributed systems, processes are independent units of execution that may require communication and synchronization across nodes. Scheduling decisions determine which node executes a process, when it starts, and how resources are allocated. This is complicated by factors such as network latency, node heterogeneity, and failure risks. Models of distributed process scheduling provide frameworks for these decisions, while metrics evaluate their effectiveness. Understanding these elements is essential for designing scalable systems that handle concurrent workloads effectively.

UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

DISTRIBUTED PROCESS SCHEDULING

LECTURE NO: -5

Distributed Systems and Process Scheduling

Distributed systems consist of multiple autonomous nodes connected via a network, sharing resources to achieve common goals. Examples include cloud platforms like AWS or Google Cloud, where tasks are distributed across virtual machines. Process scheduling in such systems extends traditional operating system scheduling (e.g., FCFS, Round-Robin) to a multi-node context.

The primary challenge is heterogeneity: nodes may differ in processing power, memory, or availability. Scheduling must account for inter-node communication costs, as data transfer can introduce delays. In concurrent programming, multiple processes run simultaneously, requiring schedulers to manage dependencies and avoid bottlenecks.

Distributed scheduling can be viewed as a resource allocation problem. It involves mapping processes to nodes while optimizing objectives like load balancing and response time. According to research, the objective is to enhance metrics such as process completion time and processor utilization. This is achieved through algorithms that consider system state, process characteristics, and network dynamics.

Scheduling is classified based on decision-making: static (pre-determined) or dynamic (runtime-adjusted). It can also be centralized (one coordinator) or decentralized (peer-based). These classifications form the basis of scheduling models, which guide implementation in real-world systems.

Models of Distributed Process Scheduling

Distributed process scheduling models define how decisions are made regarding process allocation, migration, and execution. They vary in complexity, adaptability, and overhead, influencing system performance.

1. **Static Scheduling Models**: Static models assign processes to nodes at compile time or before execution, based on predefined information. They assume predictable workloads and node capabilities. For example, in graph-based scheduling, processes are represented as directed acyclic graphs (DAGs), where nodes are tasks and edges

are dependencies. Algorithms like List Scheduling prioritize tasks by criticality and assign them to the least loaded processor.

Advantages include low runtime overhead, as decisions are made offline. However, they lack flexibility for dynamic environments, such as varying loads or node failures. In concurrent systems, static models suit embarrassingly parallel tasks, like scientific simulations, where predictability is high.

2. **Dynamic Scheduling Models**: Dynamic models adjust assignments at runtime based on current system state. They monitor load, node availability, and process progress to make real-time decisions. A common approach is load balancing, where overloaded nodes migrate processes to underloaded ones.

   Sub-models include:

   - **Sender-Initiated**: Overloaded nodes search for underloaded ones to offload tasks.
   - **Receiver-Initiated**: Underloaded nodes poll for tasks from overloaded ones.
   - **Symmetric**: Combines both, reducing probing overhead.

   Dynamic models excel in unpredictable environments, like web servers with fluctuating traffic. In concurrent programming, they support adaptive concurrency, where threads are redistributed to balance CPU usage. However, they incur overhead from monitoring and migration.

3. **Centralized Scheduling Models**: In centralized models, a single scheduler node collects global system information and makes all allocation decisions. This is similar to a master-slave architecture, where the master assigns tasks to slaves.

   For instance, in Hadoop's YARN (Yet Another Resource Negotiator), a central Resource Manager allocates resources based on cluster state. Advantages include optimal decisions from global visibility and ease of implementation. Drawbacks are scalability issues and single-point-of-failure risks. In concurrent systems, centralized models simplify synchronization but can become bottlenecks for large-scale distributions.

4. **Decentralized Scheduling Models**: Decentralized models distribute decision-making among multiple schedulers or nodes, avoiding a central authority. Each node makes local decisions based on partial information, often using gossip protocols or peer-to-peer communication.

Examples include distributed hash tables (DHTs) in systems like Chord, where tasks are hashed to nodes. In auction-based models, nodes bid for tasks based on their load. These models enhance fault tolerance and scalability, as no single failure disrupts the system. In concurrent programming, they support massively parallel environments, like in MPI (Message Passing Interface) applications, where processes negotiate resource use.

5. **Hierarchical Scheduling Models**: Hierarchical models combine centralized and decentralized approaches in a tree-like structure. Higher-level schedulers oversee groups of nodes, while lower-level ones handle local allocations.

   For example, in grid computing, a meta-scheduler coordinates multiple local schedulers. This balances global optimization with local autonomy. Advantages include better scalability than pure centralized models and more coordination than decentralized ones. In concurrent systems, hierarchical models manage nested concurrency, such as threads within processes across clusters.

6. **Hybrid and Advanced Models**:

1. Hybrid models blend static and dynamic elements, starting with static assignments and adjusting dynamically. Advanced variants incorporate machine learning for predictive scheduling, forecasting loads based on historical data. In energy-aware models, scheduling minimizes power consumption under constraints like response time.

   These models are evolving with cloud-native technologies, like Kubernetes, where schedulers use affinity rules for pod placement. In concurrent programming, they enable adaptive resource allocation for multi-threaded applications in distributed settings.

   Overall, the choice of model depends on system requirements: static for predictable workloads, dynamic for variability, centralized for simplicity, and decentralized for robustness.

Metrics for Evaluating Distributed Scheduling

Metrics quantify the effectiveness of scheduling models, guiding improvements and comparisons. They measure performance, efficiency, and fairness in distributed environments.

1. **Response Time**: Response time is the duration from process submission to completion, including waiting, execution, and communication delays. In distributed systems, it includes network latency. Lower response time indicates better scheduling. For concurrent processes, it reflects how well the scheduler minimizes blocking.
2. **Throughput**: Throughput measures the number of processes completed per unit time. High throughput signifies efficient resource use. In distributed scheduling, it accounts for parallel execution across nodes. Metrics like processes per second help evaluate scalability.
3. **CPU/Processor Utilization**: This metric tracks the percentage of time processors are active. Ideal scheduling maximizes utilization while avoiding overload. In heterogeneous systems, it's averaged across nodes. Low utilization signals poor load balancing, a common issue in concurrent distributed programs.
4. **Load Balancing Metrics**: Load balance is assessed by variance in node loads (e.g., CPU usage or queue lengths). Metrics like standard deviation of load or imbalance ratio (max load / avg load) quantify even distribution. Effective scheduling minimizes imbalance, preventing hotspots in concurrent systems.
5. **Turnaround Time**: Turnaround time is the total time from submission to completion, including waiting. It's similar to response time but emphasizes batch processing. In distributed contexts, it includes migration overheads.
6. **Waiting Time**: Waiting time is the duration a process spends in queues before execution. Minimizing it improves fairness and efficiency, especially in concurrent environments with shared resources.
7. **Fairness**: Fairness ensures equitable resource allocation among processes or users. Metrics include Jain's Fairness Index (1 for perfect fairness) or proportional share deviation. In distributed systems, it prevents starvation in concurrent workloads.
8. **Energy Consumption**: Emerging metric in green computing, measuring power used for execution. Scheduling models optimize for low energy under performance constraints, relevant for battery-powered or data-center environments.
9. **Reliability and Fault Tolerance Metrics**: These include mean time to failure (MTTF) or recovery time. In distributed scheduling, they evaluate how well the model handles node crashes, ensuring concurrent processes resume without data loss.
10. **Overhead Metrics**: Scheduling overhead measures time spent on decision-making, monitoring, and migration. Low overhead is crucial for real-time concurrent systems.

Evaluation often uses simulation tools like CloudSim to compute these metrics under various workloads. For instance, dynamic models may excel in throughput but incur higher overhead than static ones.

Challenges and Future Trends

Distributed process scheduling faces challenges like network variability, which affects metrics like response time. Heterogeneity complicates models, requiring adaptive algorithms. Concurrency introduces synchronization overheads, risking deadlocks.

Future trends include AI-driven scheduling, using machine learning for predictive metrics. Edge computing demands lightweight decentralized models. Quantum and serverless paradigms will redefine metrics, focusing on qubit utilization or function invocation times.

UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

STATIC AND LOAD BALANCING SCHEDULING

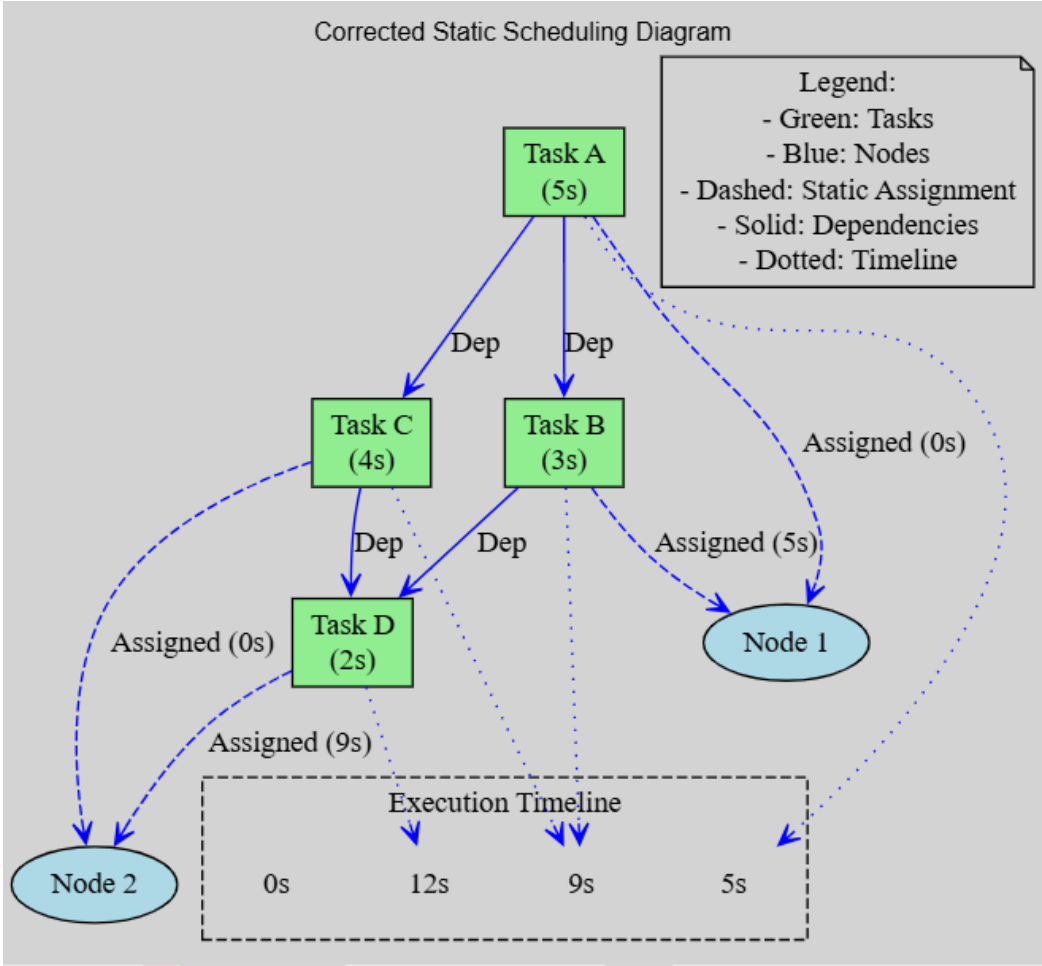
LECTURE NO: -6

Static Scheduling in Distributed Systems

Static scheduling is a foundational approach in distributed process scheduling, where the assignment of tasks or processes to nodes (processors or machines) in a distributed system is determined prior to the execution of the program. This method relies on a priori knowledge of the system's configuration, task characteristics, and dependencies, making it a non-adaptive strategy that does not adjust during runtime. In the context of concurrent programming and distributed systems, static scheduling is particularly useful for applications where predictability and low overhead are prioritized over flexibility.

Key Characteristics of Static Scheduling

- **Pre-Execution Decision Making**: All scheduling decisions are made offline, often during the compilation phase or at system startup. This involves analyzing the task graph, node capabilities, and estimated execution times to create a fixed mapping.
- **Task Representation**: Tasks are typically modeled using a **Directed Acyclic Graph (DAG)**, where vertices represent individual tasks (processes or threads), and directed edges indicate dependencies or communication requirements. For example, if Task B depends on the output of Task A, the scheduler ensures Task A completes before Task B starts, possibly assigning them to the same node to minimize data transfer latency.
- **No Runtime Overhead**: Unlike dynamic methods, static scheduling avoids continuous monitoring of system state, which reduces computational costs but assumes a stable environment.
- **Deterministic Behavior**: The schedule is fixed, leading to reproducible performance under identical conditions, which is beneficial for debugging concurrent programs.



How Static Scheduling Works

The process begins with gathering static information:

1. **Task Profiling**: Estimate execution times, communication volumes, and resource requirements for each task. This can be derived from historical data or profiling tools.
2. **System Modeling**: Assess node heterogeneity, including CPU speed, memory, and network bandwidth between nodes.
3. **Algorithm Application**: Use heuristic or optimization algorithms to assign tasks.
   - **List Scheduling**: Tasks are prioritized in a list based on criteria like the longest path (critical path) or bottom-level priority. The scheduler assigns the highest-priority ready task to the node that allows the earliest completion time, considering communication costs.
   - **Critical Path Method (CPM)**: Identifies the longest sequence of dependent tasks (critical path) and schedules them to minimize total makespan (overall completion time).
   - **Genetic Algorithms or Branch-and-Bound**: For optimal solutions in smaller systems, these explore assignment combinations to minimize metrics like makespan or cost.

Example: In a distributed rendering system for computer graphics, static scheduling might assign frame computation tasks to nodes based on their GPU power. If Node 1 has a high-end GPU, it gets complex frames, while Node 2 handles simpler ones, with the schedule fixed to avoid runtime disruptions.

In concurrent programming, static scheduling integrates with tools like OpenMP or MPI, where directives specify parallel regions, and the scheduler pre-allocates threads to nodes.

Advantages of Static Scheduling

Efficiency in Predictable **Environments**: Ideal for workloads with known characteristics, such as scientific simulations or batch processing, where it achieves near-optimal performance without runtime penalties.

- **Reduced Complexity**: Simplifies implementation in concurrent systems by avoiding synchronization issues associated with dynamic migrations.
- **Resource Optimization**: By considering all factors upfront, it can minimize communication overhead, which is crucial in distributed systems with high latency networks.
- **Energy Savings**: In static setups, nodes can be powered optimally, as assignments don't change.

Disadvantages of Static Scheduling

- **Inflexibility**: Fails to adapt to runtime variations, such as node failures or unexpected load spikes, potentially leading to poor performance.
- **Assumption Risks**: Relies on accurate estimates; errors in profiling can result in imbalances.
- **Scalability Limitations**: For large systems with thousands of nodes, computing an optimal static schedule becomes computationally infeasible (NP-hard problem).
- **Underutilization**: If a node finishes early, it remains idle until the next scheduled task, wasting resources in concurrent scenarios.

Applications

Static scheduling is widely used in high-performance computing (HPC) clusters for tasks like weather modeling or molecular dynamics simulations, where tasks are parallelized across nodes with fixed mappings. In concurrent programming, it's applied in embedded systems or real-time applications, ensuring deadlines are met through predetermined thread assignments.

Load Sharing and Load Balancing Strategies in Distributed Systems

Load sharing and load balancing are dynamic techniques aimed at distributing workload (processes or tasks) across nodes in a distributed system to achieve optimal performance. While load sharing ensures that no node is idle when work is available (focusing on utilization), load balancing goes further by striving for an equal distribution of load to minimize variance. These strategies are essential in concurrent programming, where multiple processes run simultaneously, and imbalances can lead to bottlenecks.

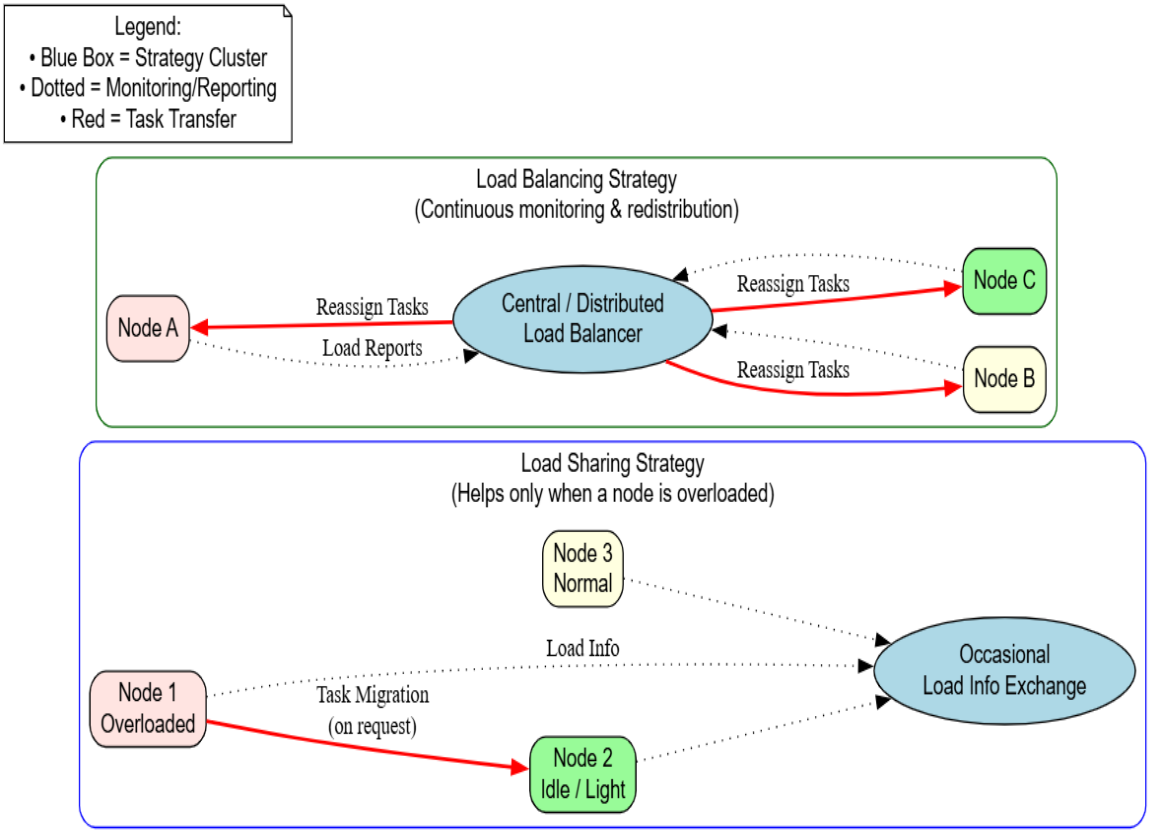
Key Characteristics of Load Sharing/Balancing

- **Dynamic Nature**: Decisions are based on runtime system state, such as CPU usage, memory availability, or queue lengths, allowing adaptation to changes.
- **Process Migration**: Central to these strategies, involving transferring running processes from overloaded to underloaded nodes, which requires checkpointing state and resuming execution.
- **Load Metrics**: Load is quantified by factors like number of processes, CPU utilization, or response time thresholds (e.g., a node is overloaded if CPU > 80%).
- **Global vs. Local View**: Strategies may use global information (centralized) or local approximations (decentralized).

How Load Sharing/Balancing Works

1. **Load Detection**: Nodes monitor their load using tools like system calls (e.g., /proc/loadavg in Linux) or distributed monitoring frameworks.
2. **Information Exchange**: Nodes share load data via:
    - Heartbeats (periodic messages).
    - Polling or broadcasting.
3. **Decision and Action**:
    - **Threshold-Based**: If load exceeds a high threshold, initiate sharing; if below low threshold, accept tasks.
    - **Migration**: Select processes for transfer based on priority or size, then migrate using virtualization (e.g., containers like Docker).
4. **Specific Strategies**:
    - **Sender-Initiated Load Sharing**: Overloaded nodes probe random or known underloaded nodes to offload tasks. Efficient when load is moderate, but probing overhead increases in high-load systems.
    - **Receiver-Initiated Load Sharing**: Underloaded nodes request tasks from overloaded ones. Better for high-load scenarios, as overloaded nodes are easier to find.

- **Symmetric Load Sharing**: Nodes act as both senders and receivers, using adaptive thresholds to balance probing. This reduces inefficiency in varying loads.
- **Centralized Load Balancing**: A coordinator (e.g., Kubernetes Scheduler) collects global load and assigns tasks using round-robin, least-loaded, or affinity rules.
- **Decentralized Load Balancing**: Peer-to-peer approaches, like work-stealing (a thread steals tasks from another's queue) or diffusion-based (load flows from high to low like heat diffusion).
- **Auction-Based**: Nodes bid for tasks based on their current load, promoting fairness.



Example: In a cloud web service like AWS Elastic Load Balancing, incoming requests are balanced across EC2 instances. If one instance is overloaded, traffic is redirected, and for processes, live migration (e.g., via VMware vMotion) balances compute loads.

In concurrent programming, these strategies integrate with frameworks like Apache Spark, where executors balance data partitions across nodes during parallel computations.

Advantages of Load Sharing/Balancing

- **Enhanced Performance**: Reduces average response time and maximizes throughput by even distribution.
- **Fault Tolerance**: Automatically redirects load from failed nodes, improving reliability in concurrent systems.
- **Scalability**: Supports elastic environments, adding/removing nodes dynamically.
- **Resource Efficiency**: Prevents hotspots, ensuring all threads and processes are productively engaged.

Disadvantages of Load Sharing/Balancing

- **Overhead Costs**: Monitoring, communication, and migration consume resources, potentially offsetting gains.
- **Thrashing Risk**: Frequent migrations in unstable systems can degrade performance.
- **Complexity**: Designing algorithms for heterogeneous nodes (different speeds) is challenging.
- **Consistency Issues**: Migration in concurrent programs may disrupt shared state, requiring careful synchronization.

Applications

Load sharing/balancing is pivotal in data centers (e.g., Google Borg scheduler), cloud computing (e.g., Azure Load Balancer), and concurrent applications like big data processing (Hadoop). In real-time systems, it ensures low-latency for concurrent user queries.

Comparison and Integration

Static scheduling is rigid and low-cost, ideal for predictable concurrent tasks, while load sharing/balancing is flexible and adaptive, suited for variable workloads. In practice, hybrid approaches combine them: static for initial allocation, dynamic for adjustments. Both enhance concurrency by optimizing resource use, but load balancing is more robust for distributed systems with unpredictable dynamics.

UNIT-3 (CONCURRENT PROCESSES AND PROGRAMMING AND DISTRIBUTED PROCESS SCHEDULING)

DISTRIBUTED FILE SYSTEMS

LECTURE NO: -7

Distributed File Systems (DFS)

Distributed File Systems (DFS) are a cornerstone of modern distributed computing, enabling multiple machines (nodes) in a network to access and share files as if they were stored on a local disk. DFS extends traditional file systems to a networked environment, allowing seamless data sharing, scalability, and fault tolerance. They are widely used in cloud storage (e.g., Google File System or GFS, Hadoop Distributed File System or HDFS), enterprise networks, and high-performance computing. Below, I'll elaborate on the transparency goals, naming mechanisms, and key characteristics of DFS, highlighting their role in achieving efficient, reliable distributed storage.

Transparency Goals in DFS

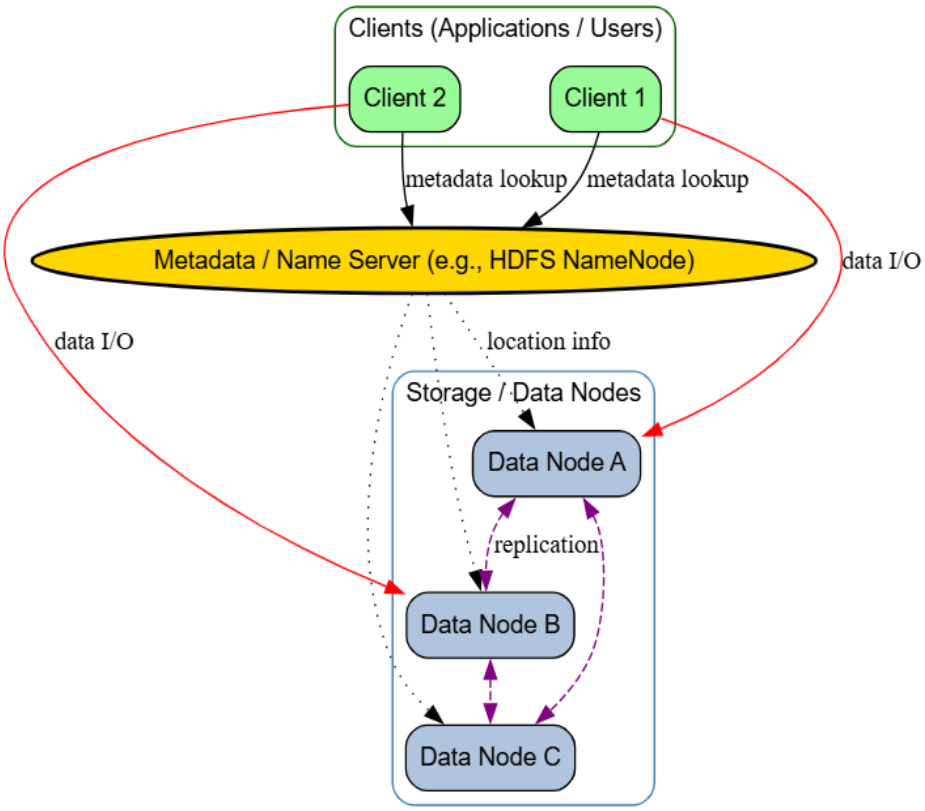
Transparency is a fundamental design principle in DFS, aiming to make the distributed nature of the system invisible to users and applications. The goal is to provide the illusion of a single, unified file system, hiding complexities like network latency, node failures, and data replication. Transparency ensures that users interact with files as they would in a local system, promoting usability and portability.

Transparency goals include:

1. **Location Transparency**: Users should not need to know the physical location of files or directories. Files appear as part of a single namespace, regardless of which node stores them. For example, in NFS (Network File System), a file path like /home/user/file.txt works the same way, whether the file is on the local machine or a remote server. This is achieved through mounting remote directories into the local file system tree.
2. **Access Transparency**: Applications access remote files using the same operations (e.g., open, read, write) as local files. The DFS handles the underlying network communication transparently. In systems like AFS (Andrew File System), caching mechanisms ensure that read/write calls feel local, with the system managing data transfer in the background.

3. **Mobility Transparency**: Users or processes can move between nodes without affecting file access. Session state or file handles are maintained across migrations. This is crucial in mobile computing or cloud environments, where virtual machines might migrate.
4. **Performance Transparency**: The system should mask variations in performance due to distribution, such as network delays. Techniques like caching (e.g., client-side caching in CIFS/SMB) and replication help maintain consistent response times, making remote access feel as fast as local.
5. **Failure Transparency**: The DFS should handle node or network failures without interrupting user access. This involves replication (storing multiple copies of files) and failover mechanisms. For instance, in GFS, data is replicated across multiple chunkservers; if one fails, the system redirects to replicas seamlessly.
6. **Replication Transparency**: Users shouldn't be aware of file replication for fault tolerance or load balancing. The DFS manages consistency (e.g., using quorum-based protocols) so that updates to one replica propagate transparently.
7. **Scaling Transparency**: As nodes are added or removed, the system should scale without users noticing changes in performance or structure. HDFS achieves this by dynamically rebalancing data blocks across new nodes.

These goals are interconnected; achieving them requires careful design, such as using metadata servers (e.g., master nodes in HDFS) to track file locations and states. However, full transparency is challenging due to inherent network issues, often leading to trade-offs like eventual consistency over strict consistency.



Naming in DFS

Naming in DFS refers to how files and directories are identified and located across the distributed environment. Unlike local file systems, DFS naming must handle distribution, ensuring unique identification, resolution, and transparency.

Key aspects of naming:

1. **Namespace Structure**: DFS typically uses a hierarchical namespace similar to Unix (e.g., /path/to/file), providing a global view. This is achieved through:
   - **Global Namespace**: A single, unified tree spanning all nodes (e.g., in Lustre or Ceph). Files are accessed via absolute paths, with metadata servers resolving locations.
   - **Mounted Namespaces**: Remote file systems are mounted into the local tree (e.g., NFS mount points), creating a logical unified view.
2. **Naming Schemes**:
   - **Location-Dependent Naming**: Names include hints about location (e.g., //server/path in SMB). This is simple but violates location transparency.

- **Location-Independent Naming**: Names are abstract, resolved via a naming service (e.g., DNS-like resolution in AFS using cell names). This enhances mobility but requires a resolution layer.

3. **Name Resolution**: The process of mapping a name to a file's location.
   - **Centralized**: A single name server (e.g., HDFS NameNode) stores metadata, resolving paths to data blocks. Fast but a single point of failure.
   - **Distributed**: Metadata is spread across nodes (e.g., in GlusterFS, using hashing to locate files). Improves scalability but increases complexity.
   - **Caching**: Clients cache resolutions to reduce latency, with invalidation mechanisms for updates.
4. **Unique Identification**: Files are identified by unique IDs (e.g., inode numbers in NFS, extended with server IDs). In replicated systems, names must resolve to the correct replica.
5. **Challenges in Naming**:
   - **Scalability**: As the system grows, name resolution must handle billions of files without bottlenecks.
   - **Consistency**: Updates to names (e.g., renames) must propagate atomically.
   - **Security**: Namespaces must support access control, often integrated with authentication (e.g., Kerberos in AFS).

Naming is pivotal for transparency; for example, in Google Cloud Storage, object names are globally unique, allowing seamless access without location awareness.

Characteristics of DFS

DFS exhibit several defining characteristics that distinguish them from local file systems, enabling distributed operation while maintaining reliability and efficiency.

1. **Distribution and Scalability**: Files are spread across multiple nodes for parallel access. DFS scale horizontally by adding nodes, with automatic data rebalancing (e.g., HDFS block placement).
2. **Replication and Fault Tolerance**: Data is replicated (e.g., 3 copies in HDFS) to survive failures. Characteristics include consistency models: strong (immediate updates) or eventual (delayed, for availability).
3. **Concurrency and Consistency**: Supports concurrent access with locking (e.g., advisory locks in NFS) or versioning. POSIX semantics (e.g., atomic writes) are often relaxed for performance.
4. **Caching and Performance**: Client-side caching reduces latency, with cache coherence protocols (e.g., lease-based in AFS) ensuring consistency.

5. **Security and Access Control**: Integrates authentication, encryption, and ACLs (Access Control Lists). For example, Kerberos tickets in AFS provide secure access.
6. **Heterogeneity**: Supports diverse hardware/OS, with abstractions for interoperability (e.g., NFSv4's cross-platform support).
7. **Metadata Management**: Central or distributed metadata (file attributes, locations) is handled separately from data for efficiency (e.g., NameNode in HDFS).

These characteristics make DFS robust for big data, cloud, and enterprise use, but they require careful implementation to balance performance and reliability.

Distributed File Systems (DFS): Design and Implementation

Distributed File Systems (DFS) are storage systems that enable multiple clients to access and share files across a network of interconnected machines, presenting a unified view of data as if it were stored locally. DFS are essential in modern computing for scalability, data sharing, and fault tolerance, supporting applications like cloud storage, big data analytics, and enterprise collaboration.

The design of DFS focuses on transparency, consistency, and performance, while implementation involves handling data distribution, replication, and network communication. Key examples include NFS (Network File System), developed by Sun Microsystems in the 1980s, and AFS (Andrew File System), created at Carnegie Mellon University. These systems exemplify different approaches to achieving distributed storage goals, with NFS emphasizing simplicity and AFS prioritizing scalability.

DFS design principles revolve around abstracting the underlying network complexities, ensuring reliable access, and optimizing for concurrent operations. Implementation typically involves client-server architectures, metadata management, and protocols for data transfer. Below, I'll elaborate on the general design and implementation, followed by overviews of NFS and AFS.

Design Principles of DFS

The design of DFS is guided by the need to manage data across heterogeneous nodes while maintaining usability and efficiency. Core principles include:

1. **Client-Server Architecture**: Most DFS use a client-server model where clients request file operations, and servers store and manage data. Servers may be centralized (e.g., a single metadata server) or distributed for scalability. Clients cache data locally to reduce latency, with mechanisms to invalidate caches on updates.

2. **Data Distribution and Replication**: Files are divided into blocks or chunks distributed across nodes for parallel access. Replication (e.g., 3 copies) ensures availability and fault tolerance. Design goals include balancing load and minimizing single points of failure.
3. **Consistency Models**: DFS must handle concurrent access. Designs range from strong consistency (immediate updates visible to all, e.g., via locking) to eventual consistency (updates propagate over time for higher availability).
4. **Caching and Prefetching**: To improve performance, designs incorporate client-side caching. Prefetching anticipates future reads, reducing network trips.
5. **Security and Access Control**: Designs integrate authentication (e.g., Kerberos) and encryption to protect data in transit and at rest.
6. **Fault Tolerance and Recovery**: Designs include heartbeat monitoring for node failures and automatic failover. Checksums verify data integrity.

These principles aim for transparency, as discussed earlier, ensuring users see a single namespace without worrying about distribution.

General Implementation Overview

Implementation of DFS involves protocols, data structures, and algorithms to realize the design:

1. **Protocols**: DFS use network protocols like RPC (Remote Procedure Call) for operations. Clients send requests (e.g., read/write) to servers, which respond with data or status.
2. **Metadata Management**: A metadata server or distributed table tracks file attributes, locations, and permissions. Implementation uses databases or in-memory structures for fast lookups.
3. **Data Storage**: Files are chunked (e.g., 64MB blocks) and stored on data nodes. Implementation includes striping (spreading chunks) for parallelism.
4. **Replication Implementation**: Servers asynchronously replicate data, using algorithms like quorum writes (majority agreement) for consistency.
5. **Caching Implementation**: Clients maintain caches with lease-based invalidation: servers grant leases, revoking them on updates.
6. **Error Handling**: Implementation includes retry mechanisms for transient failures and reconstruction for lost data (e.g., from replicas).

Tools like logging and versioning aid recovery. Scalability is implemented via sharding metadata or using consistent hashing for node assignment.

NFS: Design and Implementation Overview

NFS, originally developed by Sun Microsystems in 1984, is a client-server protocol for sharing files over a network. Its design emphasizes simplicity, portability, and interoperability across Unix-like systems. NFS versions (v2, v3, v4) evolved to improve performance and security.

Design Principles:

- **Stateless Protocol (Early Versions)**: NFS v2/v3 are stateless, meaning servers don't track client sessions, simplifying crash recovery but limiting consistency.
- **Transparency**: Provides access transparency via mounting; location is somewhat transparent through file handles.
- **Idempotent Operations**: Operations are designed to be repeatable without side effects, aiding reliability over unreliable networks.
- **Caching**: Client-side caching with attribute timeouts for consistency.
- **Security**: Early versions use UNIX permissions; v4 adds ACLs and Kerberos integration.

NFS optimizes for small files and metadata operations, using UDP/TCP for transport.

Implementation Overview:

- **Architecture**: Clients mount remote directories via the mount protocol, creating a virtual file system (VFS) layer. The NFS client kernel module handles requests.
- **File Handles**: Unique identifiers (opaque to clients) reference files, including inode numbers and device IDs. Servers generate handles for lookup.
- **RPC-Based**: Uses Sun RPC for operations like LOOKUP, READ, WRITE. Clients send RPC calls; servers execute and respond.
- **Caching**: Clients cache file data and attributes, validating with GETATTR calls. Write-through or delayed writes handle updates.
- **Replication/Fault Tolerance**: Not built-in; relies on external tools like RAID. v4 adds pNFS for parallel data access.
- **Performance**: Asynchronous I/O and read-ahead prefetching reduce latency. Implementation in kernels (e.g., Linux nfsd) ensures efficiency.

NFS is implemented in operating systems like Linux, Solaris, and Windows (via Services for Unix). Its simplicity makes it widely adopted, but statelessness can lead to stale data issues.

AFS: Design and Implementation

AFS, developed at Carnegie Mellon University in the 1980s as part of the Andrew Project, is designed for large-scale, wide-area networks with thousands of users. It prioritizes scalability, security, and whole-file caching. AFS evolved into OpenAFS.

Design Principles:

- **Scalability**: Supports global namespaces for up to 7000+ clients, with cells (administrative domains) for organization.
- **Whole-File Caching**: Clients cache entire files, reducing server load and enabling offline access.
- **Strong Consistency**: Callback-based invalidation ensures clients see updates promptly.
- **Security**: Kerberos authentication and ACLs provide fine-grained access control.
- **Location Transparency**: Volumes (logical partitions) can migrate without changing names.

AFS focuses on read-heavy workloads, with optimistic concurrency for writes.

Implementation Overview:

- **Architecture**: Cell-based, with volume location databases (VLDB) tracking storage locations. Clients interact with file servers via RPC.
- **Naming and Volumes**: Files in volumes (e.g., /afs/cell/path), resolved by Ubik (distributed database) for VLDB queries.
- **Caching**: Clients use local disk caches; servers issue callbacks to invalidate on changes. Fetch-on-demand loads files.
- **Replication**: Read-only volumes replicate for availability; Ubik ensures quorum-based consistency.
- **Implementation Details**: OpenAFS uses Rx RPC protocol for communication. Clients maintain tokens (from Kerberos) for authentication. Write operations use close-to-open semantics: changes are committed on file close.
- **Fault Tolerance**: Rx handles retransmissions; volume cloning allows quick recovery.

AFS is implemented in user-space daemons (e.g., afsd on clients) and kernel modules for integration. Its cache manager (Venus in early designs) handles local operations efficiently.

Comparison of NFS and AFS

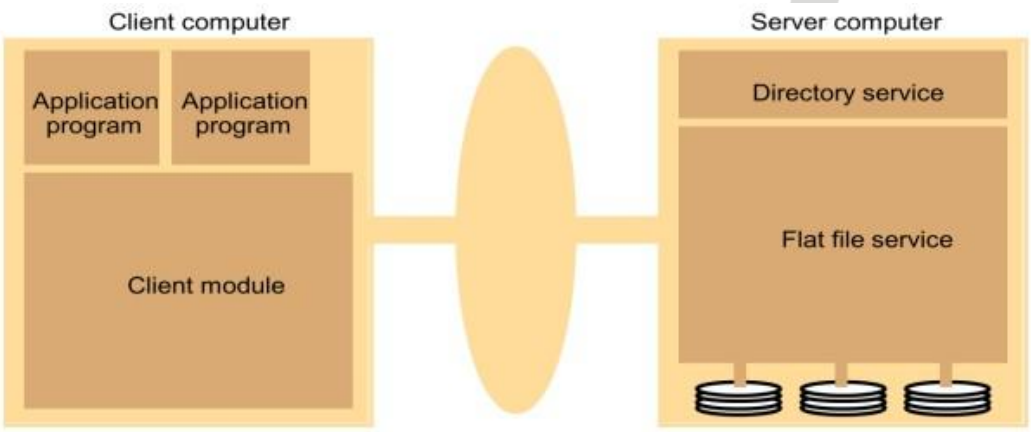
NFS is lightweight and stateless, suitable for local networks, while AFS is stateful and scalable for wide-area use. NFS offers better write performance but weaker consistency; AFS excels in caching and security but has higher overhead. Both influence modern DFS like Ceph or Lustre.

In conclusion, DFS design prioritizes transparency and scalability, with implementations balancing performance and reliability. NFS and AFS illustrate trade-offs, guiding current systems in handling distributed data challenges.

File Service Architecture

- An architecture that offers a clear separation of the main concerns in providing access to files is obtained by structuring the file service as three components:
  - A flat file service
  - A directory service
  - A client module.

- The relevant modules and their relationship is shown in Figure



- The Client module implements exported interfaces by flat file and directory services on server side.
- Responsibilities of various modules can be defined as follows:
  - Flat file service:
    - Concerned with the implementation of operations on the contents of file. Unique File Identifiers (UFIDs) are used to refer to files in all requests for flat file service operations. UFIDs are long sequences of bits chosen so that each file has a unique among all of the files in a distributed system.
  - Directory service:
    - Provides mapping between text names for the files and their UFIDs. Clients may obtain the UFID of a file by quoting its text name to directory service. Directory service supports functions needed generate directories, to add new files to directories.
  - Client module:
    - It runs on each computer and provides integrated service (flat file and directory) as a single API to application programs. For example, in UNIX hosts, a client module emulates the full set of Unix file operations.
    - It holds information about the network locations of flat-file and directory server processes; and achieve better performance through implementation of a cache of recently used file blocks at the client.

- Access control
  - In distributed implementations, access rights checks have to be performed at the server because the server RPC interface is an otherwise unprotected point of access to files.
- Directory service interface
  - Figure contains a definition of the RPC interface to a directory service.

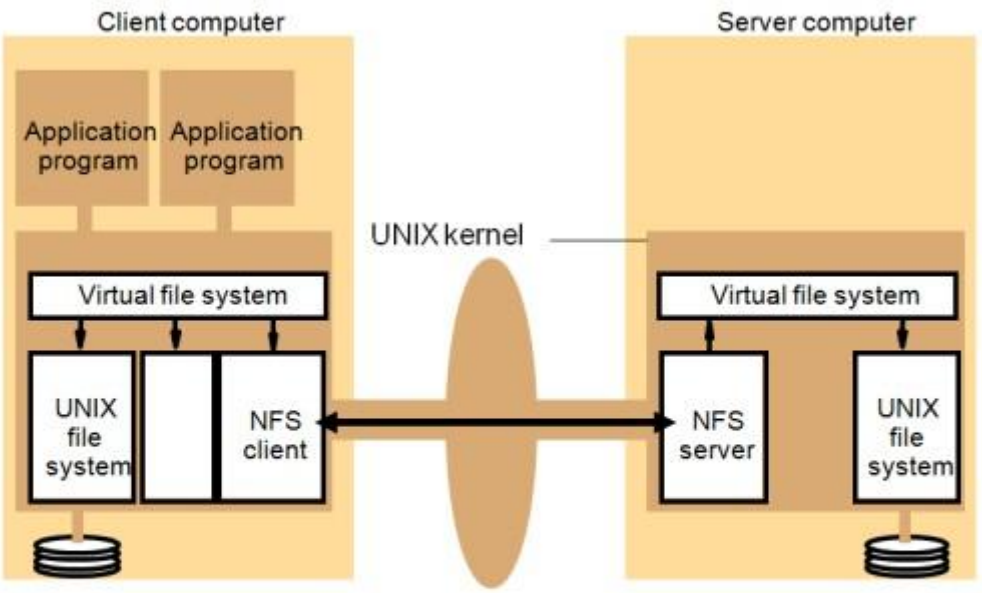
Lookup(Dir, Name) -> FileId -throws NotFound	Locates the text name in the directory and returns the relevant UFID. If *Name* is not in the directory, throws an exception.
AddName(Dir, Name, File) -throws NameDuplicate	If *Name* is not in the directory, adds(*Name,File*) to the directory and updates the file's attribute record. If *Name* is already in the directory: throws an exception.
UnName(Dir, Name)	If *Name* is in the directory, the entry containing *Name* is removed from the directory. If *Name* is not in the directory: throws an exception.
GetNames(Dir, Pattern) -> NameSeq	Returns all the text names in the directory that match the regular expression *Pattern*.

- Hierarchic file system
  - A hierarchic file system such as the one that UNIX provides consists of a number of directories arranged in a tree structure.
- File Group
  - A file group is a collection of files that can be located on any server or moved between servers while maintaining the same names.
    - A similar construct is used in a UNIX file system.
    - It helps with distributing the load of file serving between several servers.
    - File groups have identifiers which are unique throughout the system (and hence for an open system, they must be globally unique).

Case Study: 1: Sun Network File System

- **NFS (Network File System)**
  - Developed by Sun Microsystems (in 1985)
  - Most popular, open, and widely used.
  - NFS protocol standardized through IETF (RFC 1813)

NFS architecture



fh = file handle:

Filesystem identifier	i-node number	i-node generation

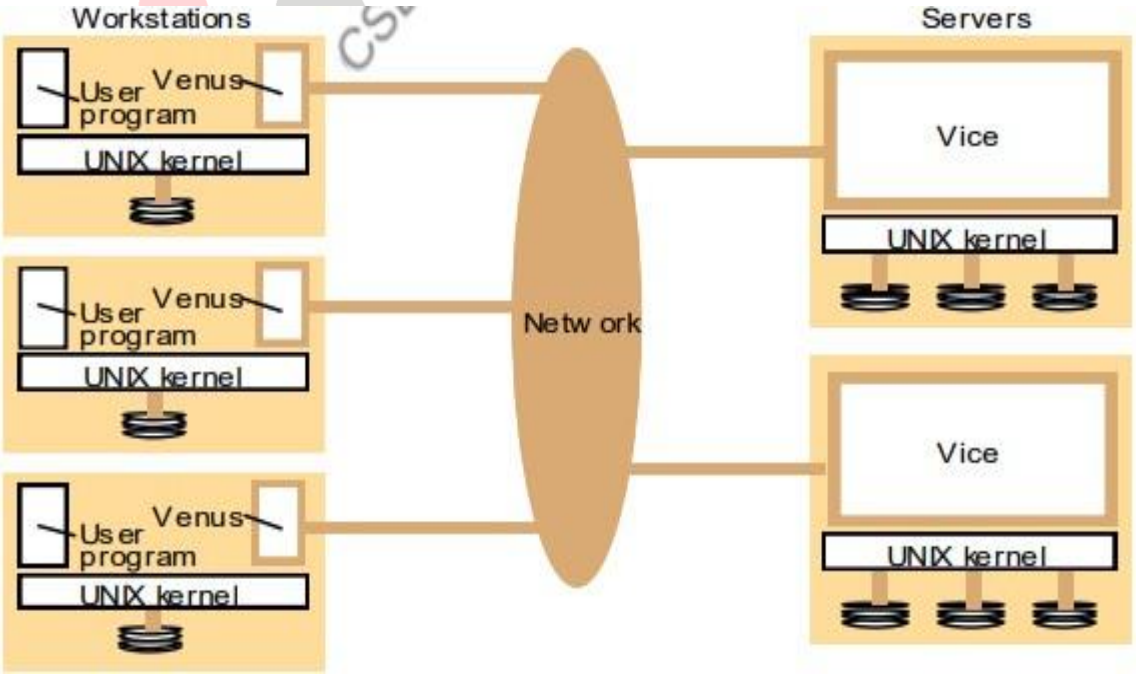
A simplified representation of the RPC interface provided by NFS version 3 servers is shown in Figure

- read(fh, offset, count) -> attr, data
- write(fh, offset, count, data) -> attr
- create(dirfh, name, attr) -> newfh, attr
- remove(dirfh, name) status
- getattr(fh) -> attr
- setattr(fh, attr) -> attr
- lookup(dirfh, name) -> fh, attr
- rename(dirfh, name, todirfh, toname)
- link(newdirfh, newname, dirfh, name)
- readdir(dirfh, cookie, count) -> entries
- symlink(newdirfh, newname, string) -> status
- readlink(fh) -> string

- mkdir(dirfh, name, attr) -> newfh, attr
- rmdir(dirfh, name) -> status
- statfs(fh) -> fsstats

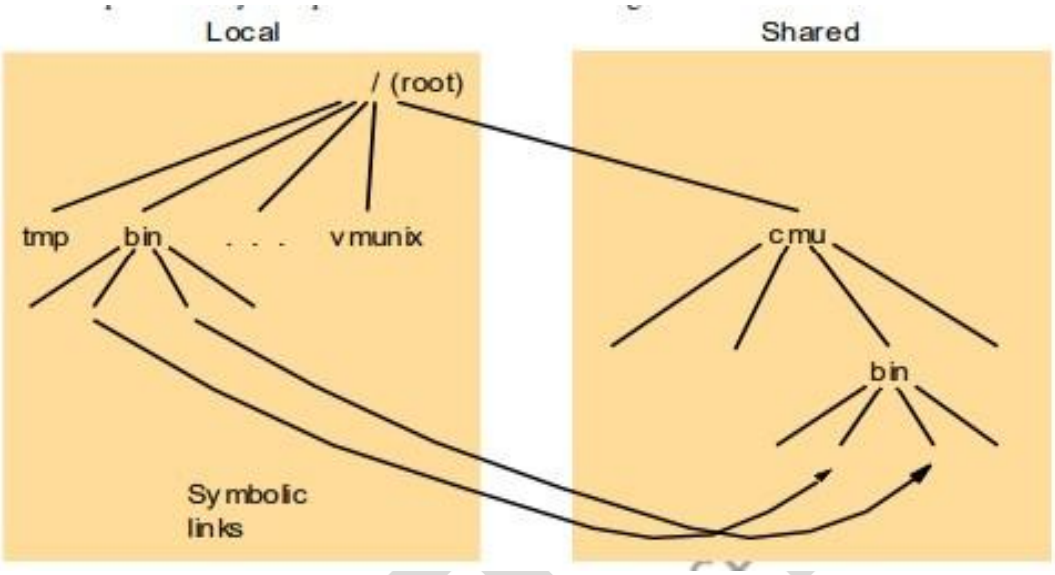
Case study 2: AFS (Andrew File System)

- Developed by Carnegie Mellon University as part of Andrew distributed computing environments (in 1986)
- A research project to create campus wide file system.
- Public domain implementation is available on Linux (LinuxAFS)
- It was adopted as a basis for the DCE/DFS file system in the Open Software Foundation (OSF, www.opengroup.org) DEC (Distributed Computing Environment
- Like NFS, AFS provides transparent access to remote shared files for UNIX programs running on workstations.
- AFS is implemented as two software components that exist at UNIX processes called Vice and Venus.
- The files available to user processes running on workstations are either local or shared.

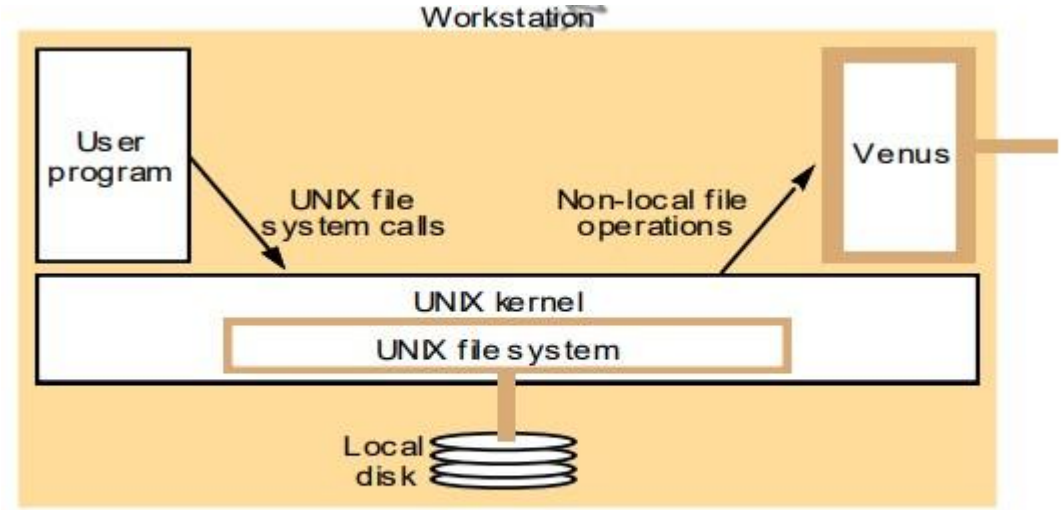


- Local files are handled as normal UNIX files.

- They are stored on the workstation's disk and are available only to local user processes.
- Shared files are stored on servers, and copies of them are cached on the local disks of workstations.
- The name space seen by user processes is illustrated in Figure



- The UNIX kernel in each workstation and server is a modified version of BSD UNIX.
- The modifications are designed to intercept open, close and some other file system calls when they refer to files in the shared name space and pass them to the Venus process in the client computer.



System call interception in AFS